

Einführung in R:

I. Konzepte und erster Einstieg

Miriam Beneke (10.11.2025) & Larissa Walter (18.11.2025)

mit Unterstützung von Franziska Henrich



Ablauf

- Einstieg
- Objekte
- Vektoren, Faktoren, Matrizen, Data Frames
- Funktionen
- Vektorisierung
- if-Bedingung, Schleifen

Begrüßung

WER SEID IHR?

Worum gehts?

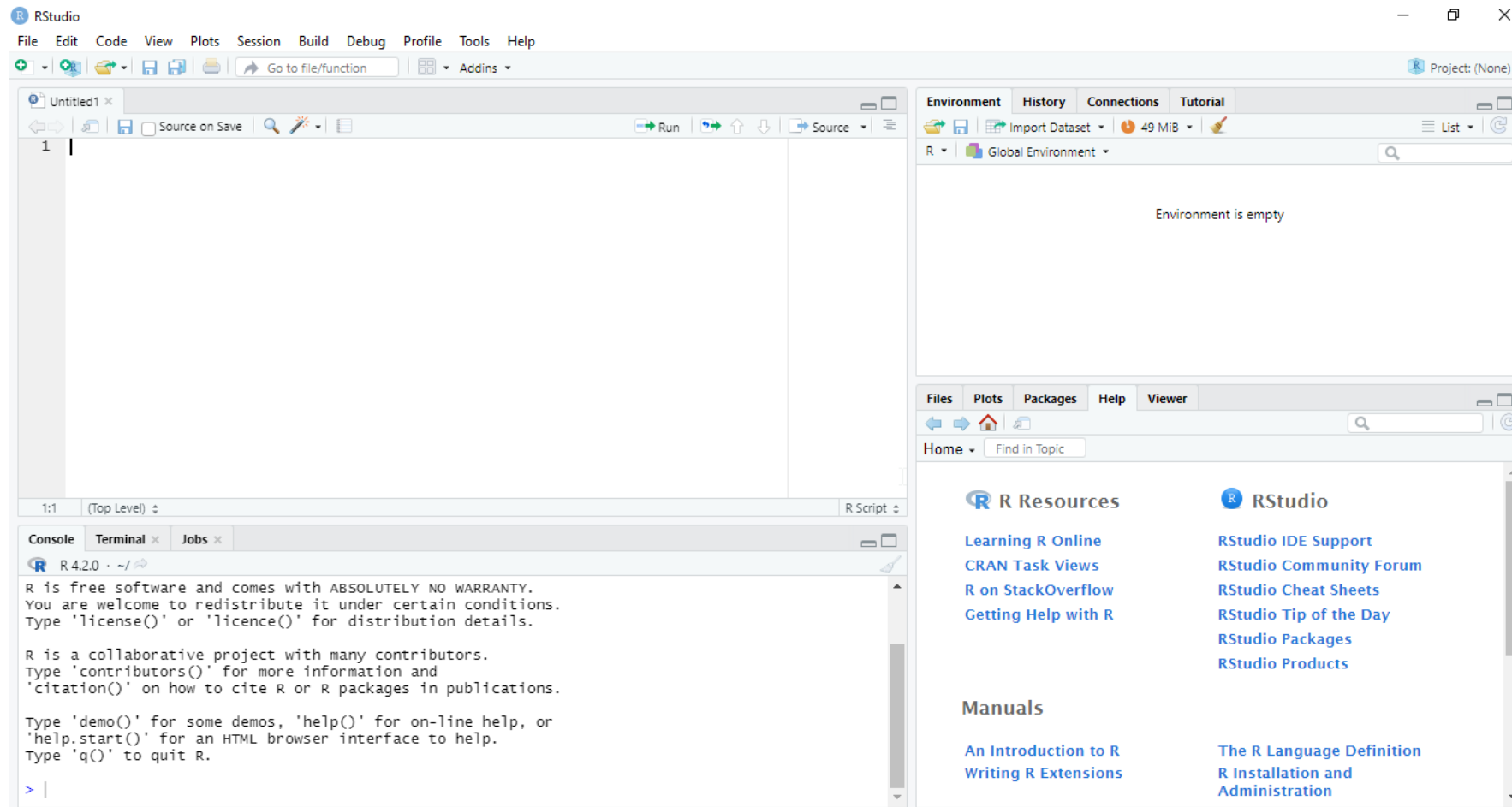
Lernen einer neuen Sprache.

- Ihr müsst die **Wörter kennen** (Vokabular)
- Ihr müsst die euch **an die Syntax halten** (Grammatik)

Ein und dieselbe Sache kann auf mehreren Wegen ausgedrückt werden (manche sind eleganter, effizienter, adäquater als andere).

So lange ihr eure Nachricht vermittelt, ohne zweideutig zu sein, sprecht ihr **fließend**.

Eins nach dem anderen



Das R Mantra - Begrifflichkeiten

John Chambers:

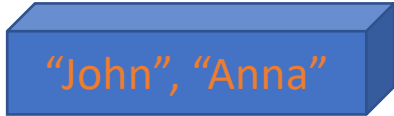
Alles was existiert ist ein Objekt.

Alles was passiert ist ein Funktionsaufruf.



Objekte

Objekte können folgendes enthalten:

- Einzelne Werte 
- Mehrere Werte 
- Andere Objekte 

Um Werte Objekten zuzuordnen, nutze <- oder =

```
object <- values(s)
```

```
object = value(s)
```

Objekte



Was sollte man bei der Benennung von Objekten beachten?

Objektnamen I

Objektnamen können folgendes enthalten:

- Buchstaben (das erste Symbol muss ein Buchstabe sein)
- Zahlen
- . und _
- keine Leerzeichen

```
object1 = 1
object_1 = 2
_object = 3    # klappt nicht
object 1 = 4   # klappt nicht
```

Tipp: Nutzt **informative** Namen mit _ (wenn nötig)

Objektnamen II

R achtet auf Groß- und Kleinschreibung:

```
x = 2
```

```
X = 2
```

R beachtet die Reihenfolge:

```
1 + z # klappt nicht
```

```
z = 2
```

Denkt dran: Um ein Objekt zu ändern, brauchen wir eine Zuweisung:

```
x + 1
```

```
## [1] 3
```

```
print(x)
```

```
## [1] 2
```

```
x = x+1
```

Objekte



Was ist die **einfachste und grundlegendste Struktur** in R?

Vektoren

Basis Objekt Struktur (außer NULL)

Beispiel: Die Antwort auf die Frage nach dem Leben, dem Universum und dem ganzen Rest:

```
the_answer = 42
```

```
the_question = „What is the answer?“
```

```
question_sensible = FALSE
```

Lasst es uns überprüfen:

```
is.vector(the_answer)
```

```
## [1] TRUE
```

Was ist sonst noch ein Vektor?

Listen

Es gibt 2 Arten von Vektoren in R:

- **atomare Vektoren**
 - enthalten Elemente desselben Typs!
 - `c()`
- **Listen**
 - enthalten Elemente von möglicherweise unterschiedlichem Typ
 - `list()`

```
is.vector(list(1, „a“, FALSE))
```

```
## [1] TRUE
```

Atomare Vektoren



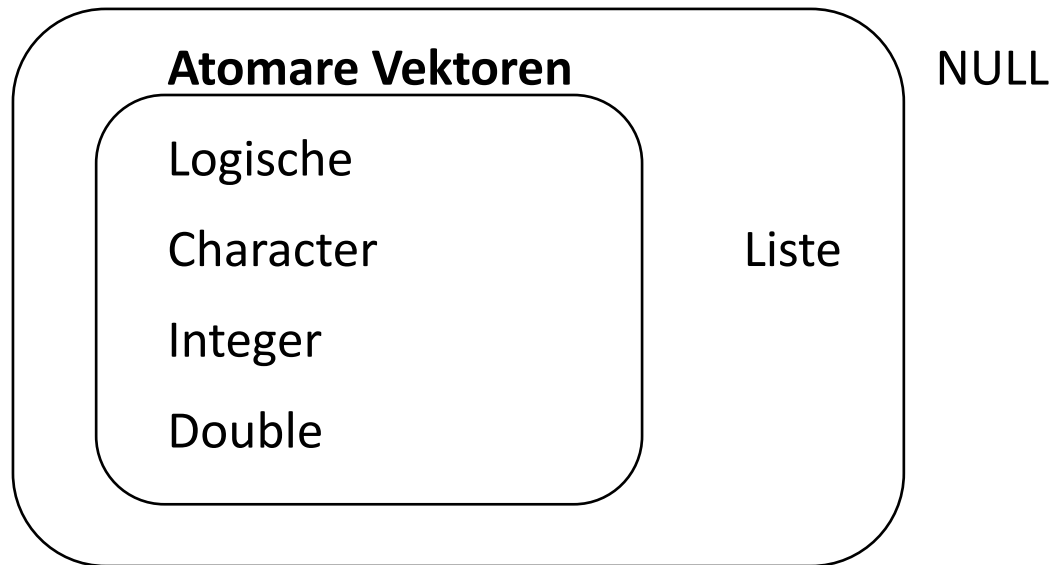
Was sind die 4 wichtigsten Typen von Atomaren Vektoren?

Welche Werte kann jeder von ihnen annehmen?

`typeof()`

Vektoren

Vektoren



Sehr hilfreiche Funktionen

- `typeof()`

Numerische Vektoren



Was ist der Unterschied zwischen `x_1` und `x_2`?

```
x_1 = c(1, 2, 3, 4)
```

```
x_2 = 1:4
```


Vektoren kombinieren

Um Vektoren zu verbinden, nutze `c()`:

```
c(c(1, 2), c(3, 4))  
## [1] 1 2 3 4
```

Achtet hier auf die Feinheiten:

```
a = list(list(1, 2), list(3, 4))  
b = list(c(1, 2), c(3, 4)) # nicht dasselbe!
```

Was passiert hier?

```
c(TRUE, 3, „covfefe“)  
## [1] „TRUE“ „3“ „cofefe“
```

Atomare Vektoren



Was wird hier passieren?

```
c(FALSE, 5)
```

Matrizen

Der klassische Weg:

```
mr_anderson = matrix(c(1, 2, 3, 4), nrow = 2, ncol = 2)
```

```
mr_anderson
```

```
##      [,1] [,2]
```

```
## [1,]    1    3
```

```
## [2,]    2    4
```

Data Frames

Wie Daten in R repräsentiert werden:

Ein `data.frame` ist eine **Liste** von **atomaren Vektoren derselben Länge** mit zusätzlichen Attributen:

- `names` (Variablen Namen)
- `class(data.frame)`
- `row.names` (Beobachtungen)
- `dimensions` (Beobachtungen, Variablen)

Objekt Strukturen: Übersicht

Dimensionen	Atomar	Nicht atomar
1d	Atomarer Vektor	Liste
2d	Matrix	Data Frame
nd	Array	

Data Frames

Fähigkeitsbeurteilung

```
harry_potter = data.frame(person = c(„harry“, „hermine“,  
                                     „ron“, „seamus“),  
                           skill = c(80, 100, 70, 42))
```

```
harry_potter  
##   person    skill  
## 1   harry     80  
## 2 hermine   100  
## 3    ron     70  
## 4  seamus    42
```

Data Frames



Aufgabe: Erstellt einen Data Frame, in dem ihr Figuren aus euren Lieblingsfilmen nach Beliebtheit bewertet.

Elementnamen

Elemente von Objekten (Listen, atomaren Vektoren, Arrays, ...) können Namen haben:

```
age = c( „Joe B“ = 79,  
        Nancy = 82,  
        Donald = 76)
```

```
age
```

```
## Joe B   Nancy   Donald
```

```
##    79     82     76
```

Namen können auch im Nachhinein definiert werden:

```
neo = list(„Mr Anderson“, 28, TRUE)
```

```
names(neo) = c(„real_name“, „age“, „the_one“)
```


Subsetting Vectors / Teilmengen extrahieren

```
object[index_vector]
```

Wir können **Positionen** nutzen, um Teilmengen zu extrahieren (**bei 1 anfangen!!**)

```
test = c(„a“, „b“, „c“)
```

```
test[1]
```

```
## [1] „a“
```

```
test[c(1, 2)]
```

```
## [1] „a“ „b“
```

```
test[-1]    # Achtung! Anderes Verhalten als in anderen Sprachen!
```

```
## [1] „b“ „c“
```

Subsetting Vectors

Nutze **names**, um Elemente zu indizieren:

```
age[„Nancy“]  
## Nancy  
##      82  
age[c(„Joe B“, „Donald“)]  
## Joe B Donald  
##      79      76
```

Logische Vektoren geben TRUE-Elemente zurück:

```
age[c(TRUE, TRUE, FALSE)]  
## Joe B Nancy  
##      79      82
```



Subsetting Vectors

Nichts gibt alle Elemente zurück:

```
age[]  
## Joe B Nancy Donald  
##      79      82      76
```

Null gibt einen Vektor mit `length() = 0` zurück:

```
age[0]  
## named numeric(0)
```

Wir können (neuen) Elementen auch neue Werte zuweisen:

```
age[„Joe B“] = 80  
age[„Kamala“] = 57
```

Ein paar Worte zu Listen

Für Listen können wir auch `[]` benutzen (dä! Sie sind Vektoren)

Allerdings neigen sie dazu, sich merkwürdig zu verhalten:

```
neo[1]
```

```
## $real_name
```

```
## [1] „Mr Anderson“
```

Das ist ein Character Vektor, oder?

```
typeof(neo[1])
```

```
## [1] „list“
```

Was ist hier passiert?

Subsetting Lists

Es gibt 3 subsetting Operatoren:

[

- kann mehrere Elemente extrahieren
- wenn es auf eine Liste angewandt wird, gibt es immer eine Liste zurück

[[

- kann nur genau ein Element extrahieren
- extrahiert einzelne Elemente aus Listen und behält den Typ bei

\$

- `x$y` ist eine Kurzform von `x[[„y“]]`

Subsetting Matrizen

Bei Matrizen spezifizieren wir die Teilmenge **explizit für jede Dimension**:

```
mr_anderson
```

```
##      [,1] [,2]
```

```
## [1,]    1    3
```

```
## [2,]    2    4
```

```
mr_anderson[1,]
```

```
## [1] 1 3
```

```
mr_anderson[2, 2]
```

```
## [1] 4
```

```
mr_anderson[1] # Moment: was ist das?!
```

```
## [1] 1
```

Subsetting Data Frames

Data Frames verhalten sich wie Listen *und* Matrizen:

- Spezifiziere einen einzelnen Vektor (mit [, [[oder \$), dann verhalten sie sich wie Listen
- Spezifiziere zwei Vektoren ([,]), dann verhalten sie sich wie Matrizen:

```
str(harry_potter[1])  
## `data.frame`: 4 obs. of 1 variable:  
## $ person: chr „harry“ „hermine“ „ron“ „seamus“  
str(harry_potter[, 1])  
## chr [1:4] „harry“ „hermine“ „ron“ „seamus“  
str(harry_potter$person)  
## chr [1:4] „harry“ „hermine“ „ron“ „seamus“
```

Unser Favorite

Matrizen



Aufgabe: Erstelle eine Matrix mit 5 Zeilen und 3 Spalten, die die Zahlen 1 bis 15 in spaltenweise absteigender Reihenfolge enthält. Erstelle eine weitere Matrix derselben Größe, die dieselben Zahlen in zeilenweise absteigender Reihenfolge enthält. Welche Zahl steht jeweils in der 2. Spalte und 4. Zeile?

Das R Mantra

John Chambers:

Alles was existiert ist ein Objekt.

Alles was passiert ist ein Funktionsaufruf.

Funktionen



Was ist eine Funktion? Und was gibt eine Funktion zurück?

Funktionen

Um eine Funktion aufzurufen, müssen wir **Argumente übergeben**.

Die Funktion macht etwas mit diesen Argumenten.

Am Ende **gibt** die Funktion etwas **zurück**.

```
function_name(ARGUMENT_1, ARGUMENT_2, ...) {  
    return_value = {<MAGIC THINGS HAPPEN>}  
    return(return_value)
```

Funktionen: Argumente

Um die Argumente einer Funktion herauszufinden, gib ein `?function_name`:

`?sum`

R: Sum of Vector Elements ▾

Find in Topic

`sum {base}`

[R Documentation](#)

Sum of Vector Elements

Description

`sum` returns the sum of all the values present in its arguments.

Usage

```
sum(..., na.rm = FALSE)
```

Arguments

`...` numeric or complex or logical vectors.

`na.rm` logical. Should missing values (including `NaN`) be removed?

Funktionen: Rückgabewerte

Rückgabewerte werden (typischerweise) in der Console ausgegeben. Um sie zu speichern, muss man den Rückgabewert einem Objekt zuweisen.

Kopiere **NIEMALS** Werte manuell in R hin und her!

```
test_results = t.test(1:4 ~ c(1, 1, 0, 0))
```

```
test_results$statistic
```

```
##          t
```

```
## 2.828427
```

```
test_results$p.value
```

```
## [1] 0.1055728
```

Vektorisierung

Mit manchen Funktionen wollen wir eine Ausgabe von einem Wert erhalten:

```
x = 1:9  
length(x)  
## [1] 9  
sum(x)  
## [1] 45  
mean(x)  
## [1] 5  
min(x)  
## [1] 1
```

Funktionen



Warum funktioniert:

```
sum(1, 2, 3, 4, 5, 6, 7, 8, 9)  
## [1] 45
```

Aber nicht:

```
mean(1, 2, 3, 4, 5, 6, 7, 8, 9)  
## [1] 1
```

Was muss geändert werden, damit der Mittelwert richtig berechnet wird?

Was passiert, wenn man `mean()` auf einen logischen Vektor anwendet?

Vektorisierung

Manchmal wollen wir eine Funktion auf *jedes* Element einzeln anwenden.

Zum Beispiel: Erhöhe jedes Element von x um 2:

```
x[1] = x[1] + 2
```

```
x[2] = x[2] + 2
```

```
x[3] = x[3] + 2
```

```
...
```

Hallo **Vektorisierung**:

```
x + 2
```

```
## [1] 3 4 5 6 7 8 9 10 11
```


Vektorisierung

R ist für Vektorisierung gemacht.

Es macht das Leben viel einfacher, aber man sollte sich dessen bewusst sein!

```
# Das Produkt von zwei Vektoren gibt keinen Skalar zurück ...
```

```
c(2, 3) * c(2, 6)
```

```
## [1] 4 18
```

```
# ... es sei denn, man fragt konkret danach:
```

```
c(2, 3) %*% c(2, 6)
```

```
##      [,1]
```

```
## [,1]    22
```

Vektorisierung

Recycling: Kürzere Vektoren werden recycled, um zu den langen zu passen (so oft hintereinander geklebt, bis er so lang ist wie gewünscht):

```
1:3 * 2
## [1] 2 4 6
1:3 * c(2, 2, 2) # genau dasselbe
## [1] 2 4 6
```

Achtung! Das kann komische Folgen haben. Was passiert hier?

```
x = c(1, 2, 3)
x[c(TRUE, FALSE)]
## [1] 1 3
```

Vektorisierung

Die meisten relationalen Operatoren sind vektorisierte Funktionen (Argument: Vektor => Rückgabewert: Vektor)

```
c(1, 2, 3) == c(1, 2, 3)
## [1] TRUE TRUE TRUE
```

Das ist sehr praktisch beim subsetting:

```
x = rnorm(30) # 30 Zufallsvariablen aus einer Standardnormalverteilung
x[x<0] = 0    # bei 0 abgeschnitten (beachte recycling!)
```

Testen, ob zwei Vektoren identisch sind (statt elementweise zu testen):

```
identical(x, y) # aber Achtung: dies ist ein sehr strenger Vergleich
```

Data Frames



Aufgabe: Erstelle einen Data Frame mit verschiedenen Obstsorten (Äpfel, Bananen, Mangos, Feigen) und entsprechenden Preisen (2.23, 1.5, 4.86, 1.25 pro Stück). Wie teuer ist dein Einkauf, wenn du für einen Familienausflug 20 Äpfel, 13 Mangos, 7 Bananen und 42 Feigen einkaufst? (Hinweis: Hier könnte die Funktion `which` hilfreich sein)

Conditionals & Schleifen

Conditionals führen bestimmte Operationen nur aus, wenn gewisse Voraussetzungen erfüllt sind.

Schleifen führen eine bestimmte Operation wiederholt aus. Sie sind sehr hilfreich, wenn etwas mehrfach in gleicher Weise wiederholt werden muss. Es gibt zwei Typen von Schleifen:

- `for` Loops führen eine Operation für eine bestimmte Anzahl an Iterationen durch
- `while` Loops führen eine Operation solange durch, wie eine bestimmte Bedingung erfüllt wird
- `repeat` führt eine Operation solange durch, bis man es unterbricht

Conditionals

Um ein Conditional statement zu formulieren, nutzen wir `if` und `else`:

```
if (CONDITION_1) {  
    # mach was  
} else if (CONDITION_2) {  
    # mach was anderes  
} else {  
    # mach noch was anderes  
}
```

Conditionals



Aufgabe: Erstelle eine Zufallsvariable x aus einer Standardnormalverteilung. Mache eine Fallunterscheidung und gib aus, ob x größer, kleiner oder gleich 0 ist. (Hinweis: `?print` und `?paste` könnten hilfreich sein)

Die for Schleife

```
for(var in seq) {  
    # hier passiert etwas  
    # jedes Mal, wenn die Schleife aufgerufen wird  
}
```

Die Schleife wird so oft ausgeführt, wie es Elemente in seq gibt (es sei denn, man unterbricht sie mit break)

Innerhalb der Schleife ist var als Variable verfügbar. Der Wert ist seq[1] im ersten Schleifenaufruf, seq[2] im zweiten, und so weiter.

Typischerweise wird var als Index genutzt.

for Schleife



Aufgabe: Erstelle einen Vektor x mit 100 zufälligen Zahlen zwischen 1 und 100. Zähle, wie häufig eine Zahl in x kleiner ist als die Zahl an der Stelle davor. Gib eine Lösung mit und eine Lösung ohne eine for Schleife an. (Hinweis: Schau mal nach ?sample)